为什么需要 Skill

- 来源: https://class.imooc.com/lesson/5286#mid=105133
- 阶段: 阶段五: 【能力封装】可复用技能
- 周: 第10周 Skill：可复用技能封装
- 模块: Skill的获取与生态
- 课节: 图文 1-1 为什么需要 Skill

1-1 为什么需要 Skill

为什么需要 Skill

> [!TIP]

阅读指南

假设这个场景：你的团队正在构建一个企业级应用，需要 AI 助手帮忙做代码审查。但你发现一个问题：

```
资深工程师的代码审查流程：
  1. 根据代码类型（API/数据库/前端）选择不同的审查策略
  2. 检查是否符合团队的技术栈规范（Python 3.10+/FastAPI/PostgreSQL）
  3. 评估安全风险（SQL注入/认证/数据验证）
  4. 分析性能瓶颈（数据库查询/内存泄漏/并发问题）
  5. 验证错误处理的完整性
  6. 给出可落地的改进建议
这是一套非常专业知识体系
不是简单的"检查清单"
```

问题是：如何让 AI 掌握这套专业知识，像资深工程师一样思考和决策？ 这就是我们需要 Skill 的原因。

> 不只是编程领域。金融分析师看财报有自己的一套判断逻辑，医生读病历有自己的一套诊断思路，教师备课有自己的一套教学设计方法——每个专业领域都有沉淀多年的隐性知识体系。而这些，是通用大模型天然不具备的。Skill 要解决的，正是这个问题。

专业知识的缺失

看一个实际例子。你的团队需要 AI 帮忙审查一段数据库查询代码：

```
# 需要审查的代码
def get_user_orders(user_id):
    query = f"SELECT * FROM orders WHERE user_id = {user_id}"
    return db.execute(query)
```

普通 AI（没有专业知识）的回答：

```
AI: "这段代码看起来没什么问题，可以正常查询订单。"

× 没有识别出 SQL 注入风险
× 没有检查是否有索引优化
× 没有考虑大数据量的分页问题
× 没有验证用户权限
```

拥有专业知识的 AI 应该这样回答：

```
严重安全风险：SQL 注入漏洞
  问题：直接拼接用户输入到 SQL 语句
  修复：使用参数化查询

性能问题：可能查询大量数据
  建议：添加分页限制，检查 user_id 是否有索引

权限问题：未验证当前用户是否有权限查看这些订单
  建议：添加权限校验逻辑

改进建议：
def get_user_orders(user_id, current_user, page=1):
    if not has_permission(current_user, user_id):
        raise PermissionError()
    query = "SELECT * FROM orders WHERE user_id = ?
             LIMIT ? OFFSET ?"
    return db.execute(query, [user_id, 20, (page-1)*20])
```

- × 普通 AI：只能做表面检查，缺乏专业判断
- ✓ 专业 AI：掌握安全最佳实践、性能优化经验、架构设计原则

传统方案为什么不行

你大概会想：“那我写一个超长的提示词，把所有规范都说明清楚不就行了？”

方案：超长提示词

```
"你是一个资深代码审查专家。审查代码时要注意：
──────────────────────────────────────────────────
1. 安全：检查SQL注入、XSS、认证授权...（200字说明）
2. 性能：数据库索引、缓存策略、并发控制...（200字说明）
3. 架构：SOLID原则、设计模式、依赖注入...（200字说明）
4. 规范：Python 3.10+、类型注解、docstring...（200字）
5. 测试：单元测试、边界情况、异常处理...（200字说明）
...（总共2000+字的详细说明）"
──────────────────────────────────────────────────
```

但这个方案有几个问题：每次都加载所有内容会消耗大量 token，而且静态文本无法包含可执行的检查逻辑，更麻烦的是难以更新和维护——改一处就要重新复制粘贴。

Skill 的思路

现在，看看 Agent Skill 是如何优雅地解决这些问题的。

Agent Skill

Skill 的诞生背景

2025年10月，Anthropic 公司发布了 Agent Skill，这是 AI 领域的一个重要突破。在此之前，AI Agent 的开发面临着一个核心挑战：如何让 AI 掌握专业领域的知识和规范。

传统方式（超长提示词、MCP Server）都存在局限：只能传递“文字”或“工具”，但无法传递“决策逻辑”和“工作流程”。

Agent Skill 的突破性设计

Agent Skill 解决了这个问题，它是一种将专业知识、工作流程和最佳实践封装成标准化模块的机制。关键特性：

```
1. 渐进式加载 (Efficient)
   ──────────────────────────────────────

   只在需要时才加载相关内容
   ✓ 减少 token 消耗
   ✓ 保持快速响应

2. 可组合 (Composable)
   ──────────────────────────────────────

   AI 自动识别并协调多个 Skill
   Skill 可以堆叠使用
   ✓ 自动协调无需手动选择

3. 跨平台 (Portable)
   ──────────────────────────────────────

   同一格式在所有 Claude 产品中通用
   ✓ Claude 应用 / API / Code
   ✓ 写一次，随处使用

4. 可执行 (Powerful)
   ──────────────────────────────────────

   支持包含可执行代码
   ✓ 复杂逻辑用代码实现更可靠
   ✓ 提高任务执行的可靠性
```

开放标准与生态影响

2025年12月，Anthropic 将 Agent Skill 公布为开放标准。这意味着任何 AI 平台都可以采用——不限于 Claude，其他平台也能使用，这推动了 AI 生态的标准化。Skill 可以跨平台复用，写一次就能随处使用，降低了学习和迁移的成本。同时，开放标准也促进了社区协作，最佳实践得以共享，AI 应用的开发速度也随之加快。

Skill 的物理形式

Agent Skill 本质上就是一个 Markdown 文件 + 可选的资源文件：

```
code-review/          # Skill 目录
  ├─ SKILL.md       # 核心文件（包含指令、流程、规范）
  ├─ checklist.md   # 额外的参考文档
  └─ script.py      # 可执行的脚本（可选）
```

AI 只需读取这个 Markdown 文件，就能掌握完整的专业知识和工作流。

Skill 的使用体验

在深入学习Skill前，我们先用同样的代码审查场景，看看用Skill 后的体验:

以下是一个Skill，先大概熟悉下他的”长相“。

SKILL.md

```
---
name: code-review
description: 专业的代码审查助手,审查代码的安全性、性能、可读性和规范性
---

# 代码审查 Skill

## 审查标准
1. **安全检查**
   - SQL 注入风险
   - XSS 漏洞
   - 敏感信息泄露

2. **性能分析**
   - 时间复杂度
   - 空间复杂度
   - 潜在性能瓶颈

3. **代码质量**
   - 可读性
   - 命名规范
   - 注释完整性

4. **边界情况**
   - 空值处理
   - 异常处理
   - 边界值测试
```

> [!TIP] 这并不是一个优秀的Skill，只是为了便于你理解，所以它非常简单。真正的Skill多达上千字。

使用效果

```
第1次使用
──────────────────────────────────────────────────────

你: "帮我审查这段代码"
    [粘贴代码]

AI: (自动识别到需要 code-review Skill)
    (加载审查标准)
    "我将从以下方面审查你的代码:
     1. 安全性检查...
     2. 性能分析...
     3. 代码质量..."

    ✓ 自动应用完整的审查流程
```

```
第2次、第3次...第N次使用
──────────────────────────────────────────────────────

你: "审查一下这段代码"

AI: (自动激活 code-review Skill)
    (每次都是相同的高标准审查)

   ✓ 无需重复说明
    ✓ 标准一致
    ✓ 不占用对话上下文
```

效果解析

这段演示体现了 Skill 的两个关键价值。第一次使用时，AI 不再需要你反复提示"请按 XX 标准审查"，而是自动识别场景并加载对应的专业知识。后续使用时，每次都是同样的高标准，不会因为对话轮次增多而降低审查质量，这些标准也不会占用用户的对话上下文。

在哪里使用 Skill

了解了 Skill 的价值后，下一个问题是：在哪里可以用到它?

Skill 最初是 Anthropic 为其 AI 编程工具 Claude Code 打造的一项功能。设计思路很明确：让 AI 在编程场景中能按需加载专业知识，而不是每次都要从零开始解释上下文。

但 Skill 的魅力在于，它很快就不再局限于 Claude Code 了。由于 Anthropic 将其公布为开放标准，其他开发工具可以按照相同的格式实现兼容支持。到今天，越来越多的主流工具都接入了 Skill 生态：

- Qoder、Trae、Cursor、OpenClaw、Hermes、Codex

Qoder

Trae

Cursor

OpenClaw

Hermes

Codex

这就意味着，你写的 SKILL.md 文件可以在不同工具间复用，不需要为每个平台重写一遍。写一次，到处能用。

下一节，我们将以 Qoder 为例，实际演示如何获取和安装一个 Skill。

■ 学点英语

中文	English	音标	说明
技能	Skill	/skɪl/	将专业知识、工作流程和最佳实践封装成标准化模块的机制
代理技能	Agent Skill	/ˈeɪdʒənt skɪl/	Skill的官方名称，强调其作为Agent能力扩展模块的定位
技能核心文件	SKILL.md	/skɪl ˈmɑːkdaʊn/	Skill的核心Markdown文件，包含YAML元数据和详细指令正文
渐进加载	Progressive Loading	/prəˈgresɪv ˈloʊdɪŋ/	Skill只在需要时才加载相关内容的机制，减少token消耗

下一节预告

了解了 Skill 的价值和适用范围后，下一节我们来盘点 Skill 的获取途径——GitHub、腾讯 SkillHub、Qoder 技能市场、MiniMax 等主流生态各有何特点，并上手安装第一个 Skill。